Dense Contexts Are Hard Contexts: Lexical Density Limits Effective Context in LLMs

Giovanni Dettori
Department of Computer Science
Politecnico di Torino
Torino, IT 10129
giovanni.dettori@polito.it

Matteo Boffa
Department of Computer Science
Politecnico di Torino
Torino, IT 10129
matteo.boffa@polito.it

Danilo Giordano
Department of Computer Science
Politecnico di Torino
Torino, IT 10129
danilo.giordano@polito.it

Idilio Drago
Department of Computer Science
University of Turin
Torino, IT 10124
idilio.drago@unito.it

Marco Mellia
Department of Computer Science
Politecnico di Torino
Torino, IT 10129
marco.mellia@polito.it

Abstract

Input length and the position of relevant information are widely cited as the primary causes of degraded LLM long-context performance. Here, we study *lexical density* – the rate at which a context introduces distinct information – as a third, largely overlooked factor that systematically reduces the effective context window of LLMs. We quantify the impact of lexical density on open-weight LLMs (9B–685B) using three "find-the-needle" style benchmarks with identical length (≈12k tokens) and controlled needle position, but increasing density of information. We observe a sharp performance collapse in higher-density benchmarks: models that are near-perfect in sparse contexts drop below 60% retrieval score on denser ones. To rule out task-type confounds, we vary and control the density within each benchmark while keeping all other properties unchanged. Reducing density generally restores performance, especially in the high-density regimes where degradation appears. These results show that effective context capacity is a function of lexical density, with direct implications for real-world LLM systems operating on compact, information-rich inputs.[1]

1 Introduction

Modern LLM applications assemble inputs from increasingly heterogeneous sources – user prompts, agent configurations, persistent memory, tool outputs, and retrieved documents [Yao et al., 2022, Packer et al., 2023, Lewis et al., 2020]. However, reliably *using* this large context remains an open problem: models often fail to incorporate task-critical evidence from their inputs, producing errors that propagate through downstream actions and are difficult to attribute. Hence, understanding *when* and *why* models fail to use their context is a prerequisite to build reliable LLM-based systems.

[1] Link to the code: https://anonymous.4open.science/r/LLM_Density-1AB7/

Preprint.

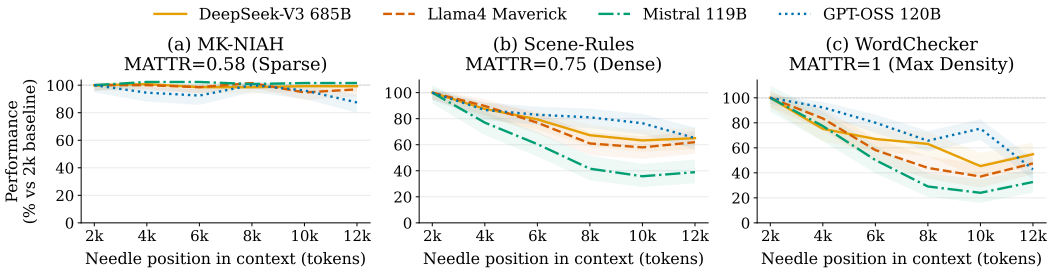


Figure 1: **Retrieval score across needle positions, normalized to the scores at the 2k position.** Shaded bands indicate 95% Wilson score intervals. All benchmarks use ≈12k-token contexts and matched needle placement, but differ in lexical density and retrieval form. The score remains high on sparse MK-NIAH, but collapses on denser Scene-Rules and WordChecker. **This motivates density as a complementary candidate axis of long-context degradation.**

Two factors are primarily known to drive context-related failures: (i) *Length*: as input grows, models become unreliable in locating relevant information, with performance often collapsing well before the advertised window [Kamradt, 2023, Hsieh et al., 2024a]; and (ii) *Position*: information in the middle of the context is recovered less reliably than at the edges (the *Lost in the Middle* effect) [Liu et al., 2024]. Both effects intensify under distractors and heavier reasoning loads [Modarressi et al., 2025, Hong et al., 2025]. However, existing evaluations implicitly assume that contexts of equal length and layout are equivalent. In this paper, we show that this is not the case, as two contexts with identical length and the same relevant information placement can show sharply different levels of difficulty. We identify a third axis explaining this discrepancy: *lexical density* – the rate at which a context introduces distinct information. Intuitively, redundant text can be skimmed, whereas dense text requires processing nearly every token.

We hypothesize that density reduces retrieval reliability at fixed length and position, shrinking the usable context window of LLMs. We quantify density using the Moving-Average Type-Token Ratio (MATTR) [Covington and McFall, 2010],[2] and test our hypothesis on *eight open-weight LLMs (9B–685B parameters)* across three "find-the-needle" benchmarks. The benchmarks have identical length (∼12k tokens), structure, and needle placement strategy, but varying density: MK-NIAH (MATTR=0.58) [Kamradt, 2023], and our two novel benchmarks SCENE-RULES (MATTR=0.75) and WORDCHECKER (MATTR=1.0, where nearly every token is new), introduced to explicitly control density.

Across benchmarks matched for length and needle placement, performance drops sharply in dense settings: models near-perfect on MK-NIAH collapse past 4k–6k tokens on Scene-Rules and WordChecker (Figure 1). This represents a failure regime an order of magnitude earlier than predicted by length alone Hsieh et al. [2024a]. Since the three benchmarks solve different tasks – exact-string, semantic, and lemma-aware – we further isolate density with within-benchmark interventions: we artificially control density while we keep task, length, and position fixed. Reducing lexical density consistently restores performance, establishing density as an additional axis of long-context degradation alongside length and position.

Contributions: (i) We identify lexical density as a previously unexplored axis in long-context evaluation and show that it systematically reduces effective context capacity. (ii) We introduce two new benchmarks, SCENE-RULES and WORDCHECKER, enabling controlled studies of density at fixed length and position. (iii) Across eight LLMs, we demonstrate that density induces performance degradation significantly earlier than predicted by length alone. (iv) Through within-benchmark interventions that hold the length, position, and task structure fixed, we show that varying lexical density produces large performance swings, ruling out length and position as sole explanations.

2 Literature Review

Long-context evaluation. The Needle-in-a-Haystack (NIAH) paradigm [Kamradt, 2023] has anchored long-context evaluation, with extensions varying task complexity [Hsieh et al., 2024a],

[2]For reference, standard prose such as *Pride and Prejudice* yields MATTR≈0.72 [Austen, 1813], while agentic configuration files such as OpenClaw Markdown Files can reach MATTR≈0.82 [Berman, 2026].

pushing context to 128k–256k tokens [Zhang et al., 2024, Yuan et al., 2025], adding document-grounded, multi-fact, or interference settings [Bai et al., 2024, Shaham et al., 2023, Kuratov et al., 2024, Xu et al., 2024], and generalizing across modalities and languages [Wang et al., 2024, Kim et al., 2025]. A growing consensus holds that NIAH is *saturated*: frontier models solve it near-perfectly regardless of length or position. The dominant response has been to change the *task* – toward application-centric [Yen et al., 2025], retrieval-and-RAG [Lee et al., 2025], or latent-structure [Vodrahalli et al., 2024] settings. We argue that saturation also stems from an overlooked property of the setup itself: NIAH haystacks are typically lexically redundant. Increasing the haystack density, while holding the retrieval task fixed, suffices to reproduce many of the failures that motivated these task-level benchmarks.

Decomposing long-context degradation. Table 1 summarizes the axes isolated by prior work; none captures haystack density. Position effects [Liu et al., 2024, Hsieh et al., 2024b] concern *where* the needle appears, not the information load around it; we confirm needle-position effects and show they are amplified by density, emerging earlier in dense settings. Levy et al. [2024] isolate length by padding a fixed reasoning sample; we make the analogous move for density, holding length and position fixed and varying only the haystack. Most closely, Modarressi et al. [2025] and Hong et al. [2025] show that reducing query–needle or query–distractor lexical overlap degrades performance. Density generalizes this from a pairwise to a haystack-level property: when *all* context items are mutually dissimilar, retrieval is hardest even at fixed query–needle similarity. The two effects are complementary.

Table 1: Axes of long-context degradation in prior work. Input **Len**gth. Needle **Pos**ition. **Q**uery–**N**eedle similarity. **Dist**ractor similarity. Haystack lexical **Den**sity, which we introduce.

	Len	Pos	Q–N	Dist	Dens
Kamradt [2023]	✓	✓			
Hsieh et al. [2024a]	✓	✓			
Liu et al. [2024]		✓			
Levy et al. [2024]	✓				
Modarressi et al. [2025]			✓		
Hong et al. [2025]				✓	
Ours		✓			✓

Information density as a prompt property. That natural language carries non-uniform information per token is classical: Shannon noted the redundancy of English [Shannon, 1951], and the Uniform Information Density literature posits that speakers modulate surface forms to keep per-token information roughly constant [Levy, 2008, Jaeger, 2010]. Prompt-compression methods operationalize this by scoring and dropping low-information tokens or chunks [Jiang et al., 2023, 2024, Li, 2023, Pan et al., 2024, Chen et al., 2025], replacing prompts with learned summaries [Mu et al., 2023, Chevalier et al., 2023], or framing compression as rate–distortion [Nagle et al., 2024]. This literature treats density as a knob to *turn down*. Our question is the inverse: when density is already high – as in agentic configurations, structured memory, or tool outputs – can the model still use the context?

Positioning. We model text-intrinsic lexical density via the Moving-Average Type–Token Ratio (MATTR) [Covington and McFall, 2010], formalized in Section 3.3. Low MATTR indicates a redundant haystack; high MATTR, one where nearly every word is new. Computed from the haystack alone – independent of query, needle, and relational structure – MATTR is complementary to position [Liu et al., 2024], task complexity [Hsieh et al., 2024a], query–evidence overlap [Modarressi et al., 2025], and distractor similarity [Hong et al., 2025]. We empirically show that lexically dense haystacks induce retrieval failures past 4k tokens where sparse haystacks remain reliable: effective context capacity depends not only on token budget, but also on lexical load.

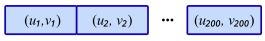
3 Methodology

3.1 Benchmarks: MK-NIAH, Scene-Rules, and WordChecker

We design *retrieval tasks* to study how lexical density affects an LLM's use of context. We cast all benchmarks in a common format. Each instance consists of a query q, a candidate set $\mathcal{C} = \{c_j\}_{j=1}^{m}$, and a unique target answer $y^\star$, the *needle*. The model receives a prompt $x = \mathrm{prompt}(q, \mathcal{C})$ and outputs an answer $\hat{y} = f_\theta(x)$, where $f_\theta(x)$ denotes the LLM's generation function. The objective is exact retrieval: the prediction is correct if $\hat{y}$ matches the target $y^\star$.[3] We denote by $|\mathcal{C}|$ the number of candidates and by $|x|$ the length of the prompt in tokens. As candidates occupy most of the context,

[3]We evaluate exact match (string match) between the $\hat{y}$ and $y^\star$. For WordChecker, both predictions and targets are normalised to their base lemmas prior to comparison.

Table 2: **Benchmark overview.** All benchmarks are cast as exact-retrieval tasks under an approximately 12k-token context budget. The table reports the query, candidate format, average candidate length $|\bar{c}|$, number of candidates $|C|$, and matching mechanism.

Benchmark	Query	Candidate Format	$\lvert\bar{c}\rvert$	$\lvert\mathcal{C}\rvert$	Matching Mechanism
MK-NIAH	Retrieve value for u_j		58	200	Exact-string (UUID)
Scene-Rules	Find r_i violating s_i		28	400	Semantic
WordChecker	Find forbidden w_j in p_i		3	4000	Lemma-aware string

we approximate the prompt length as $|x| \approx |\bar{c}| \cdot |\mathcal{C}|$, where $|\bar{c}|$ is the average candidate length in tokens. Across all benchmarks, we fix context length ($|x| \approx 12$k tokens), candidate structure, and retrieval format, while varying lexical density and needle position. Unlike prior work using up to 256k tokens [Zhang et al., 2024, Yuan et al., 2025], we operate at ≈12k tokens, a regime not expected to induce degradation from length alone [Hsieh et al., 2024a]. We describe the three benchmarks below, summarise details in Table 2, and report examples in Appendix A.

Multi-Key Needle-in-a-Haystack (MK-NIAH). MK-NIAH is the hardest variant of the standard Needle-in-a-Haystack task [Hsieh et al., 2024a, Yen et al., 2025]. The candidate set consists of user–value records, $\mathcal{C} = \{(u_j, v_j)\}_{j=1}^{m}$, where both u_j and v_j are UUIDs (35 character-long alphanumeric strings). The query q specifies one user ID. The model must locate the record whose user field matches the query and return the corresponding value. Specifically, there is a unique index $j^\star$ such that $u_{j^\star}$ matches the queried ID. The target answer is the associated value $y^\star = v_{j^\star}$. The usage of UUIDs prevents semantic shortcuts and requires retrieval from the provided context. We saturate the approximately 12k-token budget setting $|\mathcal{C}| = 200$, as each user–value averages 58 tokens.

Scene-Rules. Scene-Rules is the first new benchmark we introduce. Starting from the Moral Stories dataset [Emelin et al., 2021], we construct scenario-rule ($S - R$) pairs where the scenario violates exactly one rule (see Appendix A). We turn this into a retrieval task by setting the query to the scene, $q = s_i$, and the candidate set to a subset of rules, $\mathcal{C} = \mathcal{R}_i \subset R$, where $r_i \in \mathcal{R}_i$ is the unique gold candidate. The model must identify which rule in $\mathcal{R}_i$ the scene violates, and retrieve the rule ($y^\star = r_i$). Unlike MK-NIAH, this task requires semantic matching between the scene and the candidate rules. Each rule accounts for 28 tokens on average; therefore, we set $|\mathcal{C}| = 400$ to saturate the 12k-token budget.

WordChecker. WordChecker is the second new benchmark we introduce, inspired by Lin et al. [2020], Boffa and You [2025]. The query q is a phrase p_i, and the candidate set is a list of forbidden words, $\mathcal{C} = W_i = \{w_j\}_{j=1}^{m}$. Exactly one candidate word $w^\star \in W_i$ appears in the phrase p_i, under lemma normalisation. The model must retrieve such a forbidden word from the candidate list ($y^\star = w^\star$). Compared with the previous benchmarks, the candidates are much more granular: words account for 3 tokens on average. Hence, we set $|\mathcal{C}| = 4000$ to fill the 12k-token budget. This task does not require high-level semantic reasoning; it is a lemma-aware string-matching problem.

The three benchmarks differ along two axes: lexical density (our object of study) and the type of query–candidate matching required (exact, semantic, and lemma-aware). In Section 3.4, we isolate the effect of density through a within-benchmark intervention that varies density while holding the matching task, context length, and needle position fixed. Because such manipulations can introduce artefacts (Section 4.3), we design benchmarks with uniform candidate structure to better control them – an assumption not satisfied by existing long-context tasks. We discuss residual artefacts in Section 5.

3.2 Sampling and Positioning the Needle

We describe how we sample distractors in $\mathcal{C}$ and how the needle is positioned. As in prior work [Kamradt, 2023, Hsieh et al., 2024a], varying the needle position isolates positional effects. Our goal is to assess whether increasing distractor density degrades performance even when length-only explanations [Liu et al., 2024] would not predict a decline.

Distractor sampling. For each query, we sample distractors from the candidate pool after excluding the needle. We use a query-specific random seed to fix both the sampled distractor set and its ordering

across all experimental conditions. This ensures that performance differences across conditions are not driven by incidental variation in which distractors are sampled.

Needle position. We define the needle position by the candidate index $j \in \{1, \ldots, |\mathcal{C}|\}$ at which the needle is inserted. Since candidates are concatenated in order and have similar average length (Appendix B), j serves as a discrete proxy for the needle's token offset in the prompt. We partition $\mathcal{C}$ into eight contiguous buckets of equal size. This granularity is feasible for all three candidate sizes, $|\mathcal{C}| \in \{200, 400, 4000\}$, and yields buckets large enough to support multiple samples per bucket. To account for randomness, we uniformly sample three insertion indices for each query within each bucket and insert the needle into those indices.

3.3 Lexical Density: Moving-Average Type-Token Ratio (MATTR)

To estimate lexical density, we use *lexical diversity*, measured by the Moving-Average Type-Token Ratio (MATTR) [Covington and McFall, 2010]. Given a token sequence $w_1, \ldots, w_N$ and a window size W, MATTR averages the proportion of unique tokens over all contiguous windows of length W. It is $\mathrm{MATTR}(W) = \frac{1}{N-W+1} \sum_{i=1}^{N-W+1} |\{w_i, \ldots, w_{i+W-1}\}|/W$.

We set $W = 100$ following common practice in corpus linguistics [Covington and McFall, 2010].[4] MATTR ranges in $[1/W, 1]$: lower values indicate more repetitive text, while higher values indicate greater lexical diversity. We use MATTR as a proxy for lexical density, assuming that greater lexical diversity correlates with more distinct information. While imperfect (e.g., near-synonyms can inflate diversity without adding information), it provides a simple and practical measure for our analysis; we discuss its limitations in Section 5.

Table 3 reports multiple lexical diversity metrics (definitions in Appendix C), all yielding the same ordering: MK-NIAH is the sparsest; Scene-Rules and WordChecker are designed to extend the range toward higher-density settings. The gap is structural: MK-NIAH consists of repeated template text (e.g., *"One of the special magic uuids for <uuid1> is <uuid2>"*), with variation only in the UUIDs, resulting in low lexical diversity.

Table 3: **Lexical-density rankings are robust across metrics.** Across MATTR windows, MTLD, entropy, and Yule's K, **WordChecker is densest**, **Scene-Rules is intermediate**, and **MK-NIAH is sparsest**.

Metric	MK-NIAH	Scene-Rules	WordChecker
MATTR $W = 50$ [↑]	0.66	0.84	1.00
MATTR $W = 100$ [↑]	**0.58**	**0.75**	**1.00**
MATTR $W = 200$ [↑]	0.54	0.64	1.00
MTLD [↑]	37.65	113.76	>16k
Shannon Entropy [↑]	7.81	8.79	11.97
Yule's K [↓]	312.30	92.01	0.01

3.4 Synthetic Density Manipulation

The benchmarks of Section 3.1 differ in density, but also in matching mechanism. To establish a link between lexical density and performance, we follow the example of Levy et al. [2024] and control density *within* each benchmark by repeating distractors. For each benchmark, we pick a number k of unique distractors, sample candidates, and repeat them until we reach $|\mathcal{C}|$ items. We then shuffle candidates to introduce randomness, and introduce the needle at the selected bucket position. Smaller k means more repetition and lower MATTR; $k = |\mathcal{C}|$ recovers the original benchmark.

The needle, its position, the prompt length, and the candidate count all remain the same – only the number of unique distractors, and therefore the lexical diversity, changes. We sweep k across the following values: $\{1, 2, 20, 50, 200\}$ for MK-NIAH, $\{1, 2, 40, 100, 400\}$ for Scene-Rules, and $\{1, 20, 400, 1000, 4000\}$ for WordChecker. We chose these values to keep the multiplicative spacing roughly comparable between benchmarks. Our manipulation can only reduce lexical density, as it relies on repetition of existing distractors and cannot introduce new lexical content; thus, already sparse benchmarks cannot be densified.

3.5 Prompt format

We use the same prompt template (Appendix A) across all benchmarks and conditions, designed to keep the framing natural and avoid steering the model toward any particular solution strategy. Each

[4] Throughout the remainder of the paper, MATTR denotes this variant with window size $W = 100$.

prompt assigns the model a brief role, describes the input, states the retrieval objective, and requires a structured JSON output containing the predicted answer, a confidence score, and a one-sentence justification. We do not provide in-context examples or step-by-step reasoning instructions; the justification field is included only to support parsing and post-hoc error analysis.

4 Results

4.1 Experiment Settings

We investigate the performance of 8 recent open-source state-of-the-art LLMs, which we list in Table 4, ranging from 9 billion to 685 billion parameters. Models come from different families, and include reasoning and non-reasoning variants from both dense and Mixture of Experts (MoE) architectures. Our goal is to assess whether density is a context-confounding factor even for last-generation LLMs.

Table 4: **Models evaluated.** The suite covers **8 recent open LLMs** across **9B–685B parameters**, mixing **reasoning** and non-reasoning models, **dense** and **MoE** architectures, and both **local** and API-based inference. MoE sizes are total/active parameters.

Model	Params	R	MoE	Run
DeepSeek-V3.2	685B	**Y**	**Y**	API
Mistral Small 4	119B / 6.5B	**Y**	**Y**	API
Llama 4 Maverick	400B / 17B	N	**Y**	API
GPT-OSS-20B	21B / 3.6B	**Y**	**Y**	Local
GPT-OSS-120B	117B / 5.1B	**Y**	**Y**	Local
Qwen3.5-9B	9B	**Y**	N	Local
Qwen3.5-27B	27B	**Y**	N	Local
Qwen3.5-397B-A17B	397B / 17B	**Y**	**Y**	API

We employ a hybrid inference strategy: we evaluate the largest architectures through commercial APIs, while we run the remaining models locally on an HPC cluster with NVIDIA A40 and H200 GPUs using vLLM [Kwon et al., 2023] in bfloat16 precision. For all models, we set the temperature to T=0.1, ensuring reproducible outputs while preserving some flexibility for multi-step reasoning. We cap output length at 4096 tokens for all models except the Qwen3.5 family where we raise the cap to 16k tokens and apply a mild repetition penalty to mitigate degenerate looping.[5] We treat this as a model-specific failure mode: task-accuracy results remain comparable, but cross-family comparisons should be interpreted with this difference in mind. We evaluate each benchmark on 100 queries; for each query we test 24 needle positions (8 buckets $\times$ 3 samples) and 5 density levels, yielding $100 \times 24 \times 5 = 12{,}000$ inferences per model per benchmark.[6]

4.2 Positional Decay Under High Density

We first evaluate the three benchmarks at maximum density ($k = |\mathcal{C}|$), systematically varying the position of the needle. Concretely, we use needle position – the canonical axis along which long-context performance degrades in prior work – as a controlled probe with fixed density. We aggregate retrieval scores into common token-position bins (2k, 4k, 6k, 8k, 10k, 12k) for cross-benchmark comparison. Table 5 details the absolute results while Figure 1 provides the relative visual summary.

The bottom row shows the key finding: All models degrade as the needle moves deeper into the context, except on MK-NIAH, where performance remains stable. Averaged across the eight models, the drop relative to the 2k baseline at the deepest bin is $+1\%$ on MK-NIAH (MATTR=0.58), -27% on Scene-Rules (MATTR=0.75), and -31% on WordChecker (MATTR=1.0). Despite their different matching mechanisms, the three benchmarks show the same trend: higher MATTR (density) produces earlier and more severe positional decay.

MK-NIAH saturates. Having the **lowest density**, retrieval scores are near-perfect across all positions, with no clear trend as the needle moves – consistent with Hsieh et al. [2024a]. At $\sim$12k tokens, MK-NIAH is too sparse to challenge any model, including the smaller ones; positional decay simply does not exist in this regime.

Scene-Rules breaks the position axis. With **medium density**, moving the needle from the first to the second bin (2–4k tokens) already produces an average 11.1% relative drop, and performance keeps decaying until plateauing around 10k. The drop is heterogeneous across models: Mistral

[5] Qwen models frequently enter repetitive reasoning traces and truncate before producing the answer with short limits.

[6] Tables report only mean values for space, but each cell aggregates between 300 and 12,000 inferences depending on the level of marginalization (positions, densities). Confidence intervals are shown in Figure 1 as a representative case.

Table 5: **Retrieval Score by needle-position** on MK-NIAH, Scene-Rules, and WordChecker. For cross-benchmark comparability, **we aggregate results into common token-position bins**. Colors encode accuracy (green = higher, red = lower). **Bold** indicates the best position for each model. The bottom row reports the **average drop** across models against the 2k baseline.

	MK-NIAH ($k = 200$)						**Scene-Rules** ($k = 400$)						**WordChecker** ($k = 4000$)					
Model	2k	4k	6k	8k	10k	12k	2k	4k	6k	8k	10k	12k	2k	4k	6k	8k	10k	12k
DeepSeek-V3.2	0.99	**0.99**	0.97	0.97	0.98	0.98	**0.98**	0.85	0.78	0.66	0.62	0.63	**0.86**	0.64	0.57	0.54	0.39	0.47
Mistral Small 4	0.98	**1.00**	**1.00**	0.99	0.99	0.99	**0.86**	0.66	0.52	0.36	0.31	0.34	**0.72**	0.56	0.36	0.21	0.17	0.23
LLama 4 Maverick	0.99	0.99	0.97	**1.00**	0.93	0.96	**0.95**	0.85	0.73	0.58	0.55	0.59	**0.88**	0.74	0.52	0.39	0.33	0.42
GPT-OSS-20B	0.99	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**0.82**	0.74	0.67	0.59	0.56	0.62	**0.91**	0.80	0.71	0.46	0.59	0.34
GPT-OSS-120B	**0.95**	0.90	0.88	0.96	0.92	0.84	**0.95**	0.83	0.79	0.77	0.73	0.62	**0.96**	0.89	0.77	0.63	0.73	0.41
Qwen3.5-9B	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**0.95**	0.87	0.85	0.81	0.82	0.82	0.44	0.34	0.35	0.36	0.43	**0.46**
Qwen3.5-27B	0.90	**0.97**	0.96	0.96	0.93	0.94	**0.98**	0.93	0.89	0.83	0.86	0.83	0.79	0.72	0.79	**0.83**	0.75	0.76
Qwen3.5-397B	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	0.99	**0.94**	0.82	0.86	0.82	0.82	0.85	0.96	0.95	0.94	**0.98**	0.92	0.93
Avg. Δ vs. 2k	—	+0.01	−0.00	+0.01	+0.01	+0.01	—	−0.11	−0.17	−0.25	−0.27	−0.27	—	−0.11	−0.19	−0.27	−0.28	−0.31

Table 6: **Density sweep at fixed length and position structure.** Lexical density is **reduced** left-to-right via the number of unique keys / rules / words: the leftmost column is the original benchmark of Section 4.2, the rightmost the most sparsified variant. Each cell averages retrieval score across needle positions. Colour encodes accuracy (green = high, red = low); **bold** marks the best density per model. The bottom row reports the **average change** across models relative to the **max-density baseline** (leftmost column) within each benchmark.

Model	**MK-NIAH** *(# Unique Keys)*					**Scene-Rules** *(# Unique Rules)*					**WordChecker** *(# Unique Words)*				
	200	50	20	2	1	400	100	40	2	1	4k	1k	400	20	1
DeepSeek-V3.2	0.98	0.98	0.98	0.98	**0.99**	0.75	0.82	0.90	**0.98**	**0.98**	0.58	0.47	0.57	0.88	**0.98**
Mistral Small 4	0.99	0.99	**1.00**	0.99	0.99	0.51	0.52	0.66	0.99	**1.00**	0.38	0.27	0.29	0.57	**0.98**
Llama 4 Maverick	0.96	0.97	**0.99**	0.97	0.87	0.71	0.66	0.69	**0.96**	0.91	0.55	0.30	0.20	0.18	**0.91**
GPT-OSS-20B	**1.00**	0.99	**1.00**	**1.00**	0.98	0.66	0.71	0.82	0.93	**0.97**	0.63	0.57	0.63	0.82	**0.94**
GPT-OSS-120B	0.91	0.96	**0.99**	0.93	0.47	0.78	0.88	0.95	**1.00**	**1.00**	0.73	0.69	0.71	0.86	**0.97**
Qwen3.5-9B	**1.00**	**1.00**	0.99	**1.00**	0.99	0.85	0.90	0.94	**0.99**	0.94	0.40	0.07	0.07	0.28	**0.47**
Qwen3.5-27B	0.94	0.97	0.96	**0.97**	0.95	0.89	0.96	0.97	**0.99**	0.94	0.77	0.47	0.78	0.73	**0.86**
Avg. Δ vs. max-density	—	+0.01	+0.02	+0.01	−0.08	—	+0.04	+0.11	+0.24	+0.23	—	−0.17	−0.11	+0.04	+0.30

Small 4, despite matching GPT-OSS-120B in parameter count and MoE architecture, loses 61% relative between 2k and 12k. Qwen models, by contrast, generate substantially longer outputs (3k tokens vs. 87 for others), which may help mitigate performance collapse under high-density conditions [Zhu et al., 2025]; even so, every Qwen variant loses ∼10 points between 2k and 4k.

WordChecker is semantically simple but search-intensive. The target is a forbidden word whose lemma appears in the query, requiring no semantic inference – yet WordChecker yields the largest average drop of the three benchmarks (−31% at 12k). Five of eight models lose 45–68% of their 2k accuracy by 12k, including large MoE systems such as GPT-OSS-120B (0.96 → 0.41) and DeepSeek-V3.2 (0.86 → 0.47); only the larger Qwen variants (27B, 397B) remain stable. This shows that long-context degradation is not driven only by reasoning difficulty: even matching a lemma against a list – about as simple as retrieval gets – collapses once the list becomes dense enough.

4.3 Decoupling Lexical Density from Task Complexity

We established that denser benchmarks break down earlier than sparser ones at fixed length. A residual concern is that the three benchmarks differ not only in lexical density but also in task and content. To rule out task complexity as the driver, we apply the within-benchmark sparsification described in Section 3: task, length, and needle position are held fixed; only the number of unique distractors k (thus density) changes. Table 6 reads max-density on the left (the original benchmark of Section 4.2) and progressively sparser variants to the right; the bottom row reports the average score change across models relative to this given-density baseline.

MK-NIAH: ceiling, with one curious artefact. MK-NIAH already sits at the sparse end of the density spectrum, so further sparsification cannot make it meaningfully easier. Models stay at ceiling for $k \geq 2$ ($|\Delta| \leq 0.02$). The only outlier appears at k=1, where the average drops by −8% – driven almost entirely by GPT-OSS-120B (−47%). Trace inspection shows that, with a single repeated UUID, the task becomes *too* easy for it: the model parses the correct identifier but returns only its numeric characters, contrary to the prompt instructions. We treat this as a degenerate-output failure rather than a density effect.

Scene-Rules: clean monotonic recovery. As we sparsify the haystack, average accuracy climbs monotonically: $+4\%$, $+11\%$, $+24\%$, $+23\%$ relative to the max-density baseline. Only lexical density changes, and each step toward sparser inputs systematically restores performance. In past works, length- and position-only frameworks would predict no change in our regime. Instead, we observe a 24% accuracy swing.

Figure 2 (top) makes the joint view explicit: at the lowest density, accuracy is flat across all needle positions; positional decay emerges only as density climbs. Density and position do not act independently – density is what *turns on* the position effect.

WordChecker: noisier, but the density signal holds. On WordChecker the recovery is non-monotonic: at moderate sparsification (unique distractors k=1k, 400) accuracy is actually *worse* than at the max-density baseline (-17%, -11%), and recovers only at k=20 and k=1 ($+4\%$, $+30\%$). Trace inspection identifies the cause. Several models adopt a two-step heuristic – deduplicate the candidate list, then iterate to find a match.

With one or two repeated distractors plus the needle, the heuristic succeeds; with heavier repetition, the deduplication step strips the needle too, after which models either abstain, fall into infinite-iteration truncation, or return a word semantically associated with the sentence but absent from the candidate list (e.g., `hike` for a sentence about skiing in the mountains). The non-monotonicity is therefore an artefact of how synthetic sparsification interacts with this strategy: the max-density baseline ($k = 4$k) still produces the worst absolute scores observed in the paper, while the maximally sparse variant ($k = 1$) recovers $+30\%$ on average. Only Qwen3.5-9B fails due to its limited size – as we later discuss. We cover cleaner sparsification protocols in Section 5.

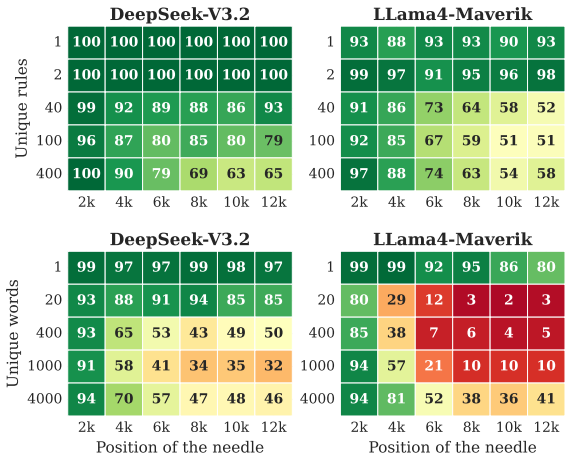


Figure 2: **Density and position interact** on Scene-Rules (top) and WordChecker (bottom). Low density flattens positional decay; high density activates it.

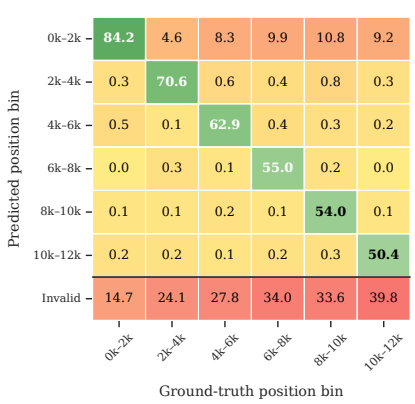
Figure 2 (bottom) provides the details: for Llama-4 Maverick, density k=20 is the worst across nearly all positions, while k=1 recovers ceiling performance; for DeepSeek-V3.2, the same non-monotonicity appears attenuated, with k=1k underperforming the k=4k baseline at deep positions.

4.4 Deconstructing the Breakdown

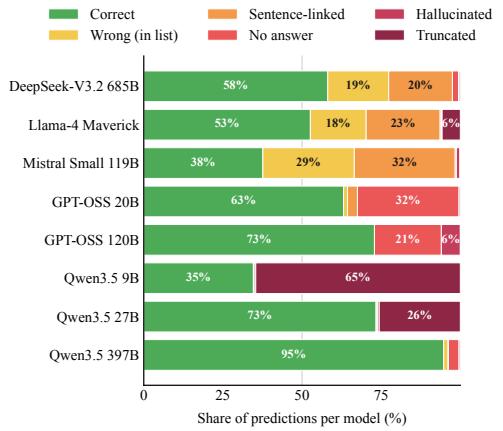
Aggregate scores tell us *that* density triggers earlier collapse; they do not tell us *how*. We use WordChecker – the maximum-density, minimum-reasoning case – as a clean playground to investigate failure modes.

Where models look (Figure 3a). We compare predicted to true needle positions, averaged across models. Two effects emerge as the needle moves deeper: i) models increasingly retrieve content from the *earliest* bins rather than from the correct one (top row); and ii) the share of invalid outputs grows with depth (bottom row). This suggests that density does not uniformly degrade attention, but biases it toward the beginning of the context.

How models fail (Figure 3b). Decomposing the predictions of each model by outcome reveals three different behavioural regimes. **(i) Conflation** (Mistral Small, Llama-4 Maverick, DeepSeek-V3.2): probability mass splits between correct answers, wrong words from the candidate list, and *sentence-linked false alarms* – declaring a forbidden word present but returning one from the sentence under validation. Both types of error grow with needle depth (Appendix E): under high density, these models conflate the two textual inputs. **(ii) Abstention** (GPT-OSS-20B, GPT-OSS-120B): both variants increasingly refuse to answer, with abstention concentrated in deeper bins. Density does not push these models toward wrong predictions – it pushes them out of the task. **(iii) Loop**



(a) WordChecker – Position of the needle: **predictions vs. ground truth**. Average of 8 models.



(b) WordChecker – **Breakdown of models' predictions**, averaged over bins.

Figure 3: **WordChecker error analysis.** Predicted vs. ground truth needle positions (left) show that higher density biases model attention toward earlier context rather than uniformly degrading it; decomposing predictions by outcome (right) reveals three qualitatively distinct failure regimes: conflation, abstention, and loop inversion.

inversion (Qwen3.5 family): reasoning traces show that the larger Qwen variants walk through sentence words and check each against the candidate list, rather than the reverse. The two procedures are logically equivalent, but keeping the long collection as the inner loop is decisive at high density: Qwen3.5-397B and Qwen3.5-27B reach 95% and 73% accuracy, while Qwen3.5-9B attempts the same strategy but truncates in $\sim$65% of cases.

The observed breakdown is not simply a stronger version of the same error as density increases. Instead, it reflects a regime change: under high-density inputs, different model families exhibit qualitatively distinct failure modes, revealing how their architectures and inference strategies cope when no tokens can be ignored.

Takeaway. On $2/3$ benchmarks, reducing lexical density at fixed length, position, and task structure produces large accuracy gains; on the third it cannot, only because the benchmark is already saturated at the sparse end. Figure 2 sharpens the picture: density does not merely lower mean accuracy – it activates and amplifies the positional decay observed in Section 4.2. Together with the controlled position sweep, these results show that lexical density is a causal contributor to long-context breakdown, interacting with position in ways that length-only explanations cannot capture.

5 Discussion and Limitations

Overall, our results show that effective context capacity depends not only on token budget and needle position, but also on the lexical density of the surrounding context. While our benchmarks enable controlled analysis of lexical density, this work has implications and limitations that define its scope and suggest directions for future research.

Implications for compression-based approaches. Many recent methods reduce context length via prompt or memory compression, but in doing so, increase lexical density. Our results highlight a potential trade-off: denser contexts can degrade retrieval performance even at moderate lengths. This raises the question of whether compression strategies, while efficient in token usage, may inadvertently reduce context reliability.

Mechanistic understanding. We characterize the behavioural impact of lexical density but do not provide a mechanistic explanation of its origin. One plausible hypothesis is that attention has limited effective resolution: in low-density contexts, redundancy allows compression over repeated spans, whereas in high-density contexts each token introduces distinct information, forcing attention to distribute more broadly and reducing retrieval precision. Preliminary results in Appendix D show

that truncating context after the needle barely improves retrieval, suggesting that distance tokens do not contribute to the "density penalty". We leave mechanistic validation to future work.

Limits of density manipulation. Our within-benchmark intervention varies density by repeating existing distractors, which can only reduce lexical diversity and cannot generate higher-density contexts. As a result, high-density regimes are studied through cross-benchmark comparisons, while within-benchmark analyses are limited to sparsification. Designing controlled methods to increase density without altering task structure remains challenging and shall be studied in future work.

Measurement of density. We measure lexical density using the Moving-Average Type–Token Ratio (MATTR), a proxy for lexical diversity. Although correlated with information load, MATTR does not capture semantic redundancy: near-synonyms or paraphrases may increase diversity without adding new information. Our findings should therefore be interpreted as effects of lexical diversity, not a complete characterization of information-theoretic density. Developing more principled measures, potentially grounded in information theory, remains an open problem.

Synthetic evaluation setting. Our experiments rely on controlled benchmarks designed to isolate density effects. While this enables causal analysis, real-world contexts often exhibit additional structure – such as topical coherence, hierarchy, or redundancy patterns – that may interact with density in ways not captured here. Preliminary evidence suggests that many practical inputs (e.g., agent configurations or technical documentation) already operate in high-density regimes, but validating these effects in naturalistic settings remains necessary.

Scope of contribution. This work is diagnostic: we identify and isolate a previously overlooked factor in long-context degradation, but do not propose mitigation strategies. Developing methods to improve robustness under high-density inputs is a natural next step.

References

Jane Austen. Pride and prejudice. `https://www.gutenberg.org/files/1342/1342-h/1342-h.htm`, 1813. Project Gutenberg eBook #1342. Last accessed: 2026-05-04.

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 3119–3137, 2024.

Matthew Berman. Openclaw: System prompt file templates (matt's markdown files). `https://gist.github.com/mberman84/663a7eba2450afb06d3667b8c284515b`, 2026. GitHub Gist. Created Feb 24, 2026. Last accessed: 2026-05-04.

Matteo Boffa and Jiaxuan You. Large-scale constraint generation–can llms parse hundreds of constraints? *arXiv preprint arXiv:2509.24090*, 2025.

Shaoshen Chen, Yangning Li, Zishan Xu, Yongqin Zeng, Shunlong Wu, Xinshuo Hu, Zifei Shan, Xin Su, Jiwei Tang, Yinghui Li, et al. Dast: Context-aware compression in llms via dynamic allocation of soft tokens. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 20544–20552, 2025.

Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3829–3846, 2023.

Michael A Covington and Joe D McFall. Cutting the gordian knot: The moving-average type–token ratio (mattr). *Journal of quantitative linguistics*, 17(2):94–100, 2010.

Denis Emelin, Ronan Le Bras, Jena D Hwang, Maxwell Forbes, and Yejin Choi. Moral stories: Situated reasoning about norms, intents, actions, and their consequences. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 698–718, 2021.

Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Technical report, Chroma, 2025. Accessed: 2026-05-01.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *First Conference on Language Modeling*, 2024a. URL https://openreview.net/forum?id=kIoBbc76Sy.

Cheng-Yu Hsieh, Yung-Sung Chuang, Chun-Liang Li, Zifeng Wang, Long Le, Abhishek Kumar, James Glass, Alexander Ratner, Chen-Yu Lee, Ranjay Krishna, et al. Found in the middle: Calibrating positional attention bias improves long context utilization. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 14982–14995, 2024b.

T Florian Jaeger. Redundancy and reduction: Speakers manage syntactic information density. *Cognitive psychology*, 61(1):23–62, 2010.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 13358–13376, 2023.

Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677, 2024.

Greg Kamradt. Needle in a haystack – pressure testing LLMs. https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023. Accessed: 2026-05-01.

Yekyung Kim, Jenna Russell, Marzena Karpinska, and Mohit Iyyer. One ruler to measure them all: Benchmarking multilingual long-context language models. In *Second Conference on Language Modeling*, 2025. URL https://openreview.net/forum?id=3vxxB3Ar9r.

Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. Babilong: Testing the limits of llms with long context reasoning-in-a-haystack. *Advances in Neural Information Processing Systems*, 37:106519–106554, 2024.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.

Jinhyuk Lee, Anthony Chen, Zhuyun Dai, Dheeru Dua, Devendra Singh Sachan, Michael Boratko, Yi Luan, Séb Arnold, Vincent Perot, Siddharth Dalmia, et al. Loft: Scalable and more realistic long-context evaluation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 6698–6723, 2025.

Mosh Levy, Alon Jacoby, and Yoav Goldberg. Same task, more tokens: the impact of input length on the reasoning performance of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15339–15353, 2024.

Roger Levy. Expectation-based syntactic comprehension. *Cognition*, 106(3):1126–1177, 2008.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33: 9459–9474, 2020.

Yucheng Li. Unlocking context constraints of llms: Enhancing context efficiency of llms with self-information-based content filtering. *arXiv preprint arXiv:2304.12102*, 2023.

Bill Yuchen Lin, Wangchunshu Zhou, Ming Shen, Pei Zhou, Chandra Bhagavatula, Yejin Choi, and Xiang Ren. Commongen: A constrained text generation challenge for generative commonsense reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1823–1840, 2020.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.

Philip Mccarthy and Scott Jarvis. Mtld, vocd-d, and hd-d: A validation study of sophisticated approaches to lexical diversity assessment. *Behavior research methods*, 42:381–92, 05 2010. doi: 10.3758/BRM.42.2.381.

Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schuetze. Nolima: Long-context evaluation beyond literal matching. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=0OshX1hiSa.

Jesse Mu, Xiang Li, and Noah Goodman. Learning to compress prompts with gist tokens. *Advances in Neural Information Processing Systems*, 36:19327–19352, 2023.

Alliot Nagle, Adway Girish, Marco Bondaschi, Michael Gastpar, Ashok Vardhan Makkuva, and Hyeji Kim. Fundamental limits of prompt compression: A rate-distortion framework for black-box language models. *Advances in Neural Information Processing Systems*, 37:94934–94970, 2024.

Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. Memgpt: towards llms as operating systems. 2023.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, et al. Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 963–981, 2024.

Uri Shaham, Maor Ivgi, Avia Efrat, Jonathan Berant, and Omer Levy. Zeroscrolls: A zero-shot benchmark for long text understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 7977–7989, 2023.

Claude E. Shannon. Prediction and entropy of printed English. *Bell System Technical Journal*, 30(1): 50–64, 1951.

Kiran Vodrahalli, Santiago Ontanon, Nilesh Tripuraneni, Kelvin Xu, Sanil Jain, Rakesh Shivanna, Jeffrey Hui, Nishanth Dikkala, Mehran Kazemi, Bahare Fatemi, et al. Michelangelo: Long context evaluations beyond haystacks via latent structure queries. *arXiv preprint arXiv:2409.12640*, 2024.

Weiyun Wang, Shuibo Zhang, Yiming Ren, Yuchen Duan, Tiantong Li, Shuo Liu, Mengkang Hu, Zhe Chen, Kaipeng Zhang, Lewei Lu, et al. Needle in a multimodal haystack. *Advances in Neural Information Processing Systems*, 37:20540–20565, 2024.

Xiaoyue Xu, Qinyuan Ye, and Xiang Ren. Stress-testing long-context language models with lifelong icl and task haystack. *Advances in Neural Information Processing Systems*, 37:15801–15840, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *NeurIPS 2022 Foundation Models for Decision Making Workshop*, 2022. URL https://openreview.net/forum?id=tvI4u1ylcqs.

Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. Helmet: How to evaluate long-context models effectively and thoroughly. In *The Thirteenth International Conference on Learning Representations*, 2025.

Tao Yuan, Xuefei Ning, Dong Zhou, Zhijie Yang, Shiyao Li, Minghui Zhuang, Zheyue Tan, Zhuyu Yao, Dahua Lin, Boxun Li, Guohao Dai, Shengen Yan, and Yu Wang. Lv-eval: A balanced long-context benchmark with 5 length levels up to 256k, 2025. URL https://arxiv.org/abs/2402.05136.

Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, et al. ∞ bench: Extending long context evaluation beyond 100k tokens. *arXiv preprint arXiv:2402.13718*, 2024.

Dawei Zhu, Xiyu Wei, Guangxiang Zhao, Wenhao Wu, Haosheng Zou, Junfeng Ran, XWang, Lin Sun, Xiangzheng Zhang, and Sujian Li. Chain-of-thought matters: Improving long-context language models with reasoning path supervision. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 3197–3211, Suzhou, China, November 2025. Association for Computational Linguistics. URL https://aclanthology.org/2025.findings-emnlp.170/.

A Benchmark details: evaluation prompts and dataset construction

This section explains the empirical framework used in our study, connecting the formal method described in Section 3 with the practical steps needed to evaluate the model. It is divided into three main parts: an analysis of cognitive load and qualitative prompt structures (Section A.1), the exact system instructions and structured output schemas used to ensure reliable parsing (Section A.2), and the multi-stage synthetic generation pipeline used to build the Scene-Rule dataset (Section A.3).

A.1 Qualitative examples and cognitive profiles

This subsection gives examples of the prompt formats tested in our study. For each benchmark, we show the query q, a sample of the context $\mathcal{C}$, and the expected answer $y^\star$. We also include a cognitive profile that explains the retrieval process in simple reasoning steps, from basic keyword matching to more complex constraint-based reasoning.

Multi-Key Needle-in-a-Haystack (MK-NIAH). MK-NIAH is our low-density baseline. It tests exact key-value retrieval in highly noisy contexts. The use of artificial UUIDs prevents semantic shortcuts, so the model must rely only on literal string matching. The prompt contains repetitive boilerplate text with many similar UUID pairs acting as distractors.

Qualitative prompt: MK-NIAH

Query (q):
What is the special magic uuid for `f488e328-[...]9f` mentioned in the following text?

Context ($\mathcal{C}$):
One of the special magic uuids for `36089374-[...]81` is: `602ba256-[...]74`.
One of the special magic uuids for `f488e328-[...]9f` is: `7c65a804-[...]c2`.
One of the special magic uuids for `9ab8180f-[...]d5` is: `ae8e6075-[...]3c`.
One of the special magic uuids for `acb8480f-[...]d5` is: `a034d075-[...]19`.

Target answer ($y^\star$): `7c65a804-[...]c2`

The model must ignore repetitive noise and find the UUID that exactly matches the query. No semantic understanding or reasoning is needed; success mainly reflects literal retrieval and positional recall over long contexts.

Scene-Rule. The Scene-Rule benchmark tests semantic understanding and deductive constraint matching in a structured setting. Unlike MK-NIAH, it prevents exact string matching. The model must match a colloquial scene to one of 400 distinct rules written in a strict, pseudo-legal style. The rules act as similar distractors, so success requires semantic alignment rather than surface-level word overlap.

Qualitative prompt: Scene-Rule

Query (q):
Scene: Jake is experiencing severe pain from a wisdom tooth removal. Despite reading the recommended dosage on his medicine, he intentionally takes double the amount.

Context ($\mathcal{C}$):
`[Id_234]` It is prohibited to inquire about another individual's medication or medical prescriptions unless explicitly invited to do so.
`[Id_343]` Individuals must not consume more than the prescribed dosage of pain medication, as indicated by a licensed healthcare provider or the medication's packaging.
`[Id_404]` It is prohibited for an individual to intentionally expose themselves to known sources of pathogens or contaminants with the purpose of inducing illness.
`[Id_416]` It is prohibited to reference one's illness in a manner that seeks to gain an advantage or manipulate others during a disagreement.

Target answer ($y^\star$): `[Id_343]`

Here, the model must connect Jake's action – intentionally taking more medicine than recommended – to Rule 343. It must also reject nearby health-related distractors, such as asking about another person's medication or using illness to manipulate an argument.

WordChecker. WordChecker tests the effect of extreme lexical density (MATTR $= 1.0$) without requiring complex reasoning. It is a lemma-aware set intersection task: the model must match words in the query to their root forms in the context. Unlike MK-NIAH, this requires partial keyword mapping rather than exact string matching. The context contains up to 4,000 unique, unstructured word distractors.

Qualitative prompt: WordChecker

Query (q):
Sentence: The students sit in a circle as the teacher talks.

Context ($\mathcal{C}$):
`discard - delete - lunch - bib - shutter - shading - sterile - cathedral - secretary - scrub - energy - polo - talk - bicycle - tradition - computer - friday - skylight - lynx - martini - [...]`

Target answer ($y^\star$): `talk`

Here, the model must keep the query in memory while scanning a dense list of unique words. It must map the surface form "talks" to the lemma "talk" and ignore thousands of unrelated distractors.

A.2 Complete evaluation prompts and structured output schemas

To make evaluation deterministic and easy to parse automatically, we require all models to return structured JSON outputs. Each model receives a system prompt containing a serialized Pydantic[7] schema. The output must include the final answer, a `reasoning` field, and a `confidence` score. The `reasoning` field is included only as a short justification to support parsing and post-hoc error analysis, not to provide in-context examples or explicitly encourage step-by-step reasoning.

MK-NIAH System Prompt. For MK-NIAH, the instruction is intentionally simple. The model is given a brief "detective" role to encourage careful extraction. The output schema requires the target UUID under the `number` key, together with the standard `confidence` and `reasoning` fields used for analysis.

System prompt template: MK-NIAH

You are a detective trying to find a secret number hidden within a text.

You will be given:
A block of text containing the hidden number.

YOUR TASK:
Scan the provided text, locate the hidden target number, and extract it exactly as it appears.

You MUST always output:
- A one-sentence explanation of where the number was found in the text.
- The extracted secret number as a string.
- A confidence score between 0.0 and 1.0.

```
{{ input_text }}
```

You MUST respond ONLY with a valid JSON object matching this exact schema:

```
{
   "number": string,    // The hidden special number.
   "confidence": float,   // Confidence score between 0.0 and 1.0.
   "reasoning": string   // One-sentence explanation of the reasoning.
}
```

[7] `https://docs.pydantic.dev/`

Scene-Rule System Prompt. For Scene-Rule, the prompt moves from string extraction to constraint matching. The model is framed as an "expert legal adjudicator" to encourage careful rule evaluation. It must compare the scenario against the candidate rules and return the exact violated rule ID (`conflicting_rule_id`). The `reasoning` field provides a short justification for the selected ID, supporting later error analysis.

System prompt template: Scene-Rule

You are an expert legal adjudicator and AI compliance judge.

You will be given:
1. A set of candidate RULES, each with a unique ID and a detailed description.
2. A SCENARIO that describes a specific situation or action. The scenario always violates one of the candidate rules.

YOUR TASK:
For each scenario, analyze the associated candidate rules and determine which rule the scenario violates.

You MUST always output:
- The ID of the broken rule.
- A confidence score between 0 and 1, where 0 means no confidence at all and 1 means absolute certainty.
- A one-sentence explanation of why you believe the scenario violates the identified rule.

```
{{ full_prompt_text }}
```

You MUST respond ONLY with a valid JSON object matching this exact schema:

```
{
   "reasoning": string,           // One sentence explaining the violation.
   "conflicting_rule_id": string,  // Exact ID of the rule (e.g., 'Id_42').
   "confidence": float            // Confidence score between 0.0 and 1.0.
}
```

WordChecker System Prompt. In WordChecker the model is framed as an "expert compliance analyst" and must find a forbidden word, or its lemma, from a short sentence within a distractor list of up to 4,000 words. The schema requires the exact `word`, ensuring the model resolves the morphological match between the sentence and the vocabulary list.

System Prompt Template: WordChecker

You are an expert compliance analyst specializing in rule adherence evaluation.

You will be given:
1. A SENTENCE
2. A list of candidate WORDS

YOUR TASK:
Identify the single word from the list that is contained within the sentence.
We say that a word is "contained" within the sentence if it appears as it is or as a lemma (e.g., "run" is contained in "running").

You MUST always output:
- The candidate words that the sentence contains.
- A confidence score between 0 and 1, where 0 means no confidence at all and 1 means absolute certainty.
- A one-sentence explanation of why you believe the scenario violates the identified rule.

```
{{ full_prompt_text }}
```

You MUST respond ONLY with a valid JSON object matching this exact schema:

```
{
   "word": string,         // The candidate word contained in the sentence.
   "confidence": float,    // Confidence score between 0.0 and 1.0.
   "reasoning": string     // One-sentence explanation of the reasoning.
}
```

A.3 Scene-Rule dataset construction

To evaluate deductive constraint satisfaction while reducing contamination and ambiguity, we build the Scene-Rule benchmark through a four-step synthetic generation pipeline:

1. **Source selection and diversity sampling.** We start from the public test split of the Moral Stories dataset [Emelin et al., 2021]. We manually remove instances containing vague or highly subjective terms, then use K-means clustering to sample diverse seed examples from the filtered set.

2. **Synthetic triplet generation.** Using these seeds, we prompt `gpt-4o` to generate triplets consisting of an objective rule, a violating scene, and a hard-negative non-violating scene. The rules are written as strict, pseudo-legal statements, avoiding subjective language and focusing on observable actions. The scenes are short narratives, with the hard negative designed to closely match the violating scene in wording and structure while not breaking the rule.

3. **Automated differential self-verification.** A separate `gpt-4o` instance verifies each triplet as a literal legal adjudicator. It evaluates both scenes against the same rule and retains the triplet only if it marks the violating scene as a breach (*True*) and the hard-negative scene as non-violating (*False*). This differential check helps ensure that success depends on semantic understanding rather than keyword overlap.

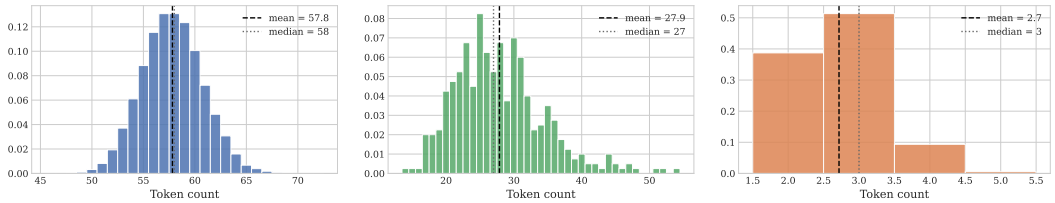
4. **Final benchmark assembly.** Triplets that fail this verification step are discarded as ambiguous. The remaining rules and scenes are then used to build the final high-density retrieval contexts, as described in Section 3.

B Candidate Distributions

This section checks whether distractor candidates have consistent lengths within each benchmark. This matters because our positional analysis uses the candidate index j as a proxy for the token position of the inserted needle, which is valid only when candidate lengths are relatively uniform.

Figure 4 shows the token-length distributions for the three benchmarks. *MK-NIAH* has the longest candidates, with a mean of 57.8 tokens and a median of 58. *Scene-Rule* has medium-length rule candidates, with a mean of 27.9 tokens and a median of 27. *WordChecker* has very short candidates, with a mean of 2.7 tokens and a median of 3.

Within each benchmark, token lengths are tightly concentrated around their medians. Therefore, ordering candidates creates an approximately linear relationship between candidate index and absolute token position.



(a) *MK-NIAH* (b) *Scene-Rule* (c) *WordChecker*

Figure 4: **Candidate token-length distributions.** Histograms show token counts for individual candidates in each benchmark. *MK-NIAH* candidates average 57.8 tokens, *Scene-Rule* candidates average 27.9 tokens, and *WordChecker* candidates average 2.7 tokens. Since lengths are tightly concentrated within each benchmark, the candidate index j is a reliable proxy for absolute token position.

C Lexical Diversity: Measure of Textual Lexical Diversity (MTLD).

We compute lexical-diversity metrics over the candidate context only, excluding fixed prompt instructions and output-schema text. Tokens are whitespace-tokenized, lowercased, and stripped of punctuation before computing MATTR.

We also measure lexical diversity using the Measure of Textual Lexical Diversity (MTLD) [Mccarthy and Jarvis, 2010]. MTLD estimates the average length of a contiguous token span over which lexical variation remains above a fixed type-token ratio (TTR) threshold. Let $w_1, \ldots, w_N$ be a token sequence and let τ denote the MTLD threshold, conventionally set to $\tau = 0.72$ [Mccarthy and Jarvis, 2010]. For a scan direction d, we move through the sequence and maintain the TTR of the current segment $w_s, \ldots, w_j$:

$$\mathrm{TTR}(s, j) = \frac{|\{w_s, \ldots, w_j\}|}{j - s + 1}. \tag{1}$$

Whenever $\mathrm{TTR}(s, j) \leq \tau$, the current segment is counted as one MTLD factor and a new segment begins at w_{j+1}. If the final segment does not reach the threshold, we add a fractional factor

$$\phi = \frac{1 - \mathrm{TTR}_{\mathrm{final}}}{1 - \tau}, \tag{2}$$

where $\mathrm{TTR}_{\mathrm{final}}$ is the TTR of the remaining segment. If F_d is the resulting number of full plus fractional factors in direction d, then

$$\mathrm{MTLD}_d = \frac{N}{F_d}. \tag{3}$$

Following standard practice, we compute MTLD in both the left-to-right and right-to-left directions and average the two scores:

$$\mathrm{MTLD} = \frac{1}{2}\left(\mathrm{MTLD}_{\rightarrow} + \mathrm{MTLD}_{\leftarrow}\right). \tag{4}$$

Larger MTLD values indicate that longer spans are needed before repetition lowers the TTR below τ, and therefore correspond to greater lexical diversity.

D Ablation study: the impact of prompt truncation and post-target noise

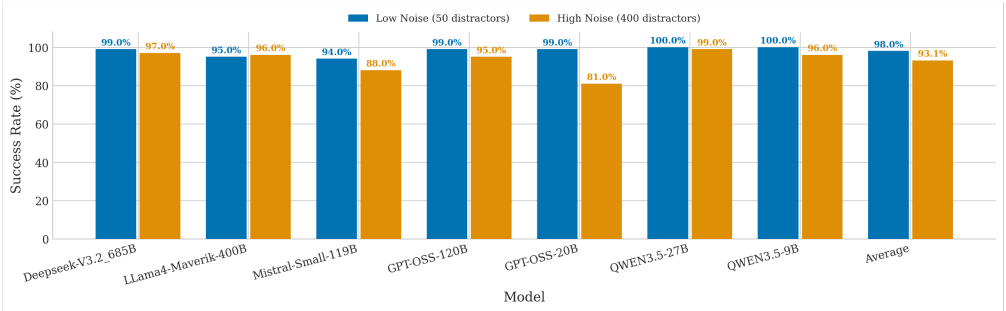
A common assumption in long-context processing is that fewer input tokens make retrieval easier, since the model has less to attend to. Removing tokens placed after the target should therefore improve performance. We test this assumption to see whether simple sorting can replace strict semantic filtering.

Experimental setup. As shown in Figure 5, we place the target near the beginning of the context and compare two conditions:

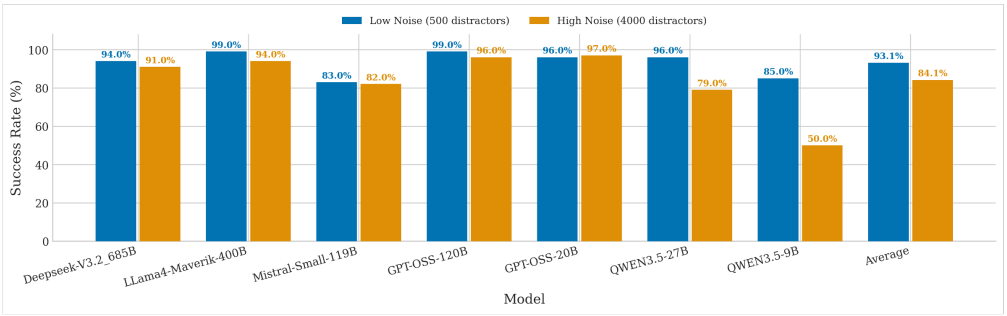
- **Truncated context (semantic filtering):** The prompt ends shortly after the target, reducing the input to about $1.5k$ tokens (50 distractors in Scene-Rule, 500 in *WordChecker*). This mimics a filter that drops irrelevant chunks.
- **Saturated context (sorting):** The target is placed early, but the prompt is padded to the full $12k$-token budget with distractors *after* the target (400 in Scene-Rule, 4000 in *WordChecker*). This mimics a system that ranks relevant chunks first but does not cut the context.

The setup checks whether a large block of tokens after the target still pulls attention away from information the model has already seen, and thus whether sorting alone is enough or filtering is strictly required.

Figure 5: **Effect of post-target noise on retrieval.** The charts compare success rates when the target is followed by a truncated prompt (filtering) versus a fully saturated $12k$-token context (sorting). Most frontier models handle the extra noise well, but a few degrade sharply under heavy distractor loads (e.g., Qwen models drop from 85% to 50% on *WordChecker*).



(a) *Scene-Rule*: truncated low-noise context (50 distractors) vs. saturated high-noise context (400 distractors).



(b) *WordChecker*: truncated low-noise context (500 distractors) vs. saturated high-noise context (4000 distractors).

Findings. Against expectations, most models cope well with downstream noise: success rates in saturated $12k$-token contexts are nearly identical to those in truncated, low-noise settings. In practice, this means simple sorting is enough for robust models. Their attention locks onto the target early and is not pulled away by later tokens, so strict semantic filtering is not needed.

There are, however, specific cases where the original intuition holds and truncation is still required. Qwen3.5 models degrade only on *WordChecker* (e.g., Qwen3.5-9B drops from 85% to 50%): error logs show that high noise triggers an "overthinking" loop in which the model echoes the context until it runs out of tokens. GPT-OSS-20B degrades only on Scene-Rule (99% to 81%), likely because its smaller size struggles to keep attention anchored when surrounded by semantically dense pseudo-legal distractors.

E Detailed Error Analysis by Needle Position

As reported in our main findings, breaking down predictions on the *WordChecker* benchmark under extreme lexical density reveals three distinct failure modes: **(i) Conflation**, where models mix candidate words with words from the query sentence; **(ii) Abstention**, where models refuse to answer; and **(iii) Loop Inversion**, where models enter a reasoning pattern that leads to output truncation.

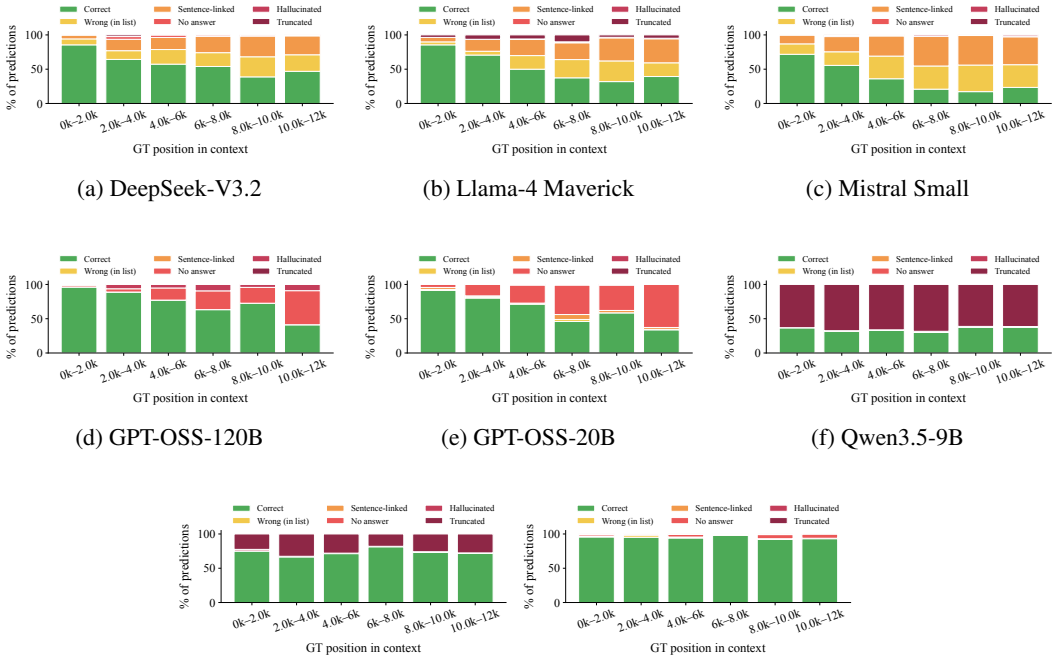
To see how these failures behave with depth, we group predictions by the ground-truth (GT) position of the target needle within the saturated $12k$-token context. Figure 6 shows this position-wise breakdown for all evaluated models.

Two observations stand out:

1. **Performance drops with depth.** As expected in needle-in-a-haystack settings, accuracy decreases the deeper the needle is placed. The share of correct predictions falls steadily across almost all models, raising the overall error rate.
2. **The type of error does not change.** While the *number* of errors grows with depth, the *kind* of error stays the same for each model. Greater depth amplifies a model's existing failure mode rather than introducing new ones.

For example, models in the **Conflation** regime (Mistral Small, DeepSeek-V3.2) lose their correct predictions at deeper bins entirely to "Wrong (in list)" and "Sentence-linked" false alarms; they do not start refusing to answer. Models in the **Abstention** regime (GPT-OSS variants) instead show a growing "No answer" block at deeper positions, without ever drifting into hallucinations or conflation. The Qwen family's **Loop Inversion** appears as a steady share of "Truncated" outputs that grows with depth as token exhaustion compounds.

Overall, the search strategies models use in hyper-dense contexts appear hardcoded: when pushed past their limits, they fail in predictable, architecture-specific ways.



(a) DeepSeek-V3.2 (b) Llama-4 Maverick (c) Mistral Small

(d) GPT-OSS-120B (e) GPT-OSS-20B (f) Qwen3.5-9B

(g) Qwen3.5-27B (h) Qwen3.5-397B

Figure 6: **Prediction outcomes by ground-truth (GT) needle position on *WordChecker*.** The stacked bars show that overall accuracy drops as the target is placed deeper in the context, but the specific failure mode (Conflation, Abstention, or Truncation) stays the same for each model across all positions.